

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 09-2

NOMBRE COMPLETO: Antonio Manuel Zamora Ayala

Nº de Cuenta: 320152896

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 3 de noviembre de 2026

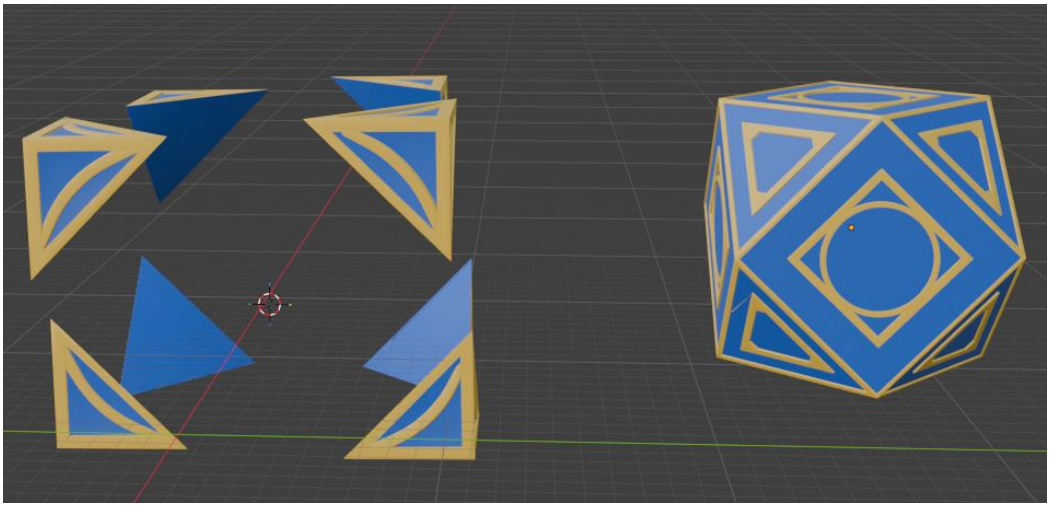
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. **Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.**

1.1 Holocron hace la animación del gif.

Primero comencé texturizando el modelo del holocron en Blender que previamente había sido separado para prácticas pasadas.



Luego exporté el holocron por partes y en *P09-2_320152896.cpp* modifiqué los bloques de código siguientes:

- **Carga del modelo:**

```
171   //Holocron
172   Model centroHolocron;
173   Model piramide1;
174   Model piramide2;
175   Model piramide3;
176   Model piramide4;
177   Model piramide5;
178   Model piramide6;
179   Model piramide7;
180   Model piramide8;
```

```
537     //Holocron
538     centroHolocron = Model();
539     centroHolocron.LoadModel("Models/cuerpoHolo.obj");
540
541     piramide1 = Model();
542     piramide1.LoadModel("Models/piramide1.obj");
543
544     piramide2 = Model();
545     piramide2.LoadModel("Models/piramide2.obj");
546
547     piramide3 = Model();
548     piramide3.LoadModel("Models/piramide3.obj");
549
550     piramide4 = Model();
551     piramide4.LoadModel("Models/piramide4.obj");
552
553     piramide5 = Model();
554     piramide5.LoadModel("Models/piramide5.obj");
555
556     piramide6 = Model();
557     piramide6.LoadModel("Models/piramide6.obj");
558
559     piramide7 = Model();
560     piramide7.LoadModel("Models/piramide7.obj");
561
562     piramide8 = Model();
563     piramide8.LoadModel("Models/piramide8.obj");
```

- **Animación compleja del holocron:** Para esta parte me apoyé de Google Gemini para que me ayudara a implementar la animación compleja, ya que al principio lo pensé mediante if/else, pero me dí cuenta que esto caía dentro de la animación simple. Le dí el siguiente prompt: “**si quisiera hacer la misma animación por medio de algoritmos o ecuaciones, ¿qué debería modificar?”**

Código inicial con animación simple:

```
// Lógica de animación del Holocron
if (estadoHolocron == 0) {
    // Fase 0: Separando
    separaPiramides += separaPiramidesOffset * deltaTime;
    if (separaPiramides >= 2.0f) {
        separaPiramides = 2.0f;
        estadoHolocron = 1;
        tiempoPausa = 0.0f;
    }
```

```
}
else if (estadoHolocron == 1) {
    // Fase 1: Pausa de 1 segundo
    tiempoPausa += deltaTime;
    if (tiempoPausa >= 1.0f) {
        estadoHolocron = 2;
    }
}
else if (estadoHolocron == 2) {
    // Fase 2: Regresando
    separaPiramides -= separaPiramidesOffset * deltaTime;
    if (separaPiramides <= 0.0f) {
        separaPiramides = 0.0f;
        estadoHolocron = 3;
        tiempoPausa = 0.0f;
    }
}
else if (estadoHolocron == 3) {
    // Fase 3: Pausa de 1 segundo antes de repetir
    tiempoPausa += deltaTime;
    if (tiempoPausa >= 1.0f) {
        estadoHolocron = 0;
    }
}
```

La animación compleja obtenida con Google Gemini es la siguiente:

```
106   // Variables para el Holocron
107
108   float rotaHolo = 0.0f;
109   float rotaHoloOffset = 0.01f;
110
111   float rotaPiramides = 0.0f;
112   float rotaPiramidesOffset = 0.005f;
113
114   float separaPiramides = 0.0f;
115   float separaPiramidesOffset = 0.01f;
116   float giroAcomodo = 0.0f;
117   float tiempoTotal = 0.0f;
```

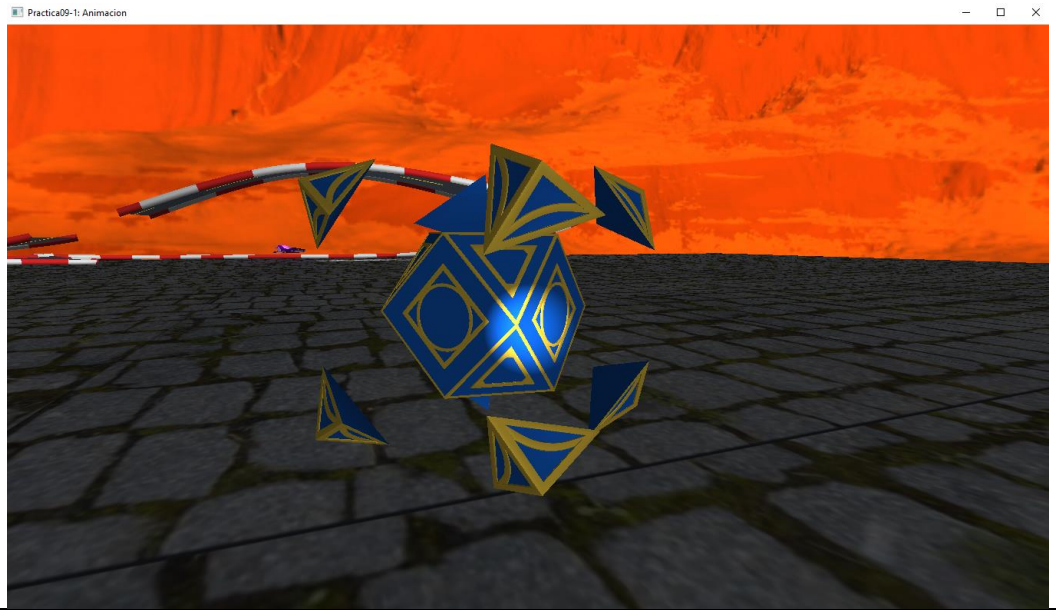
```
939       // Animación para el holocorn
940
941       rotaHolo += rotaHoloOffset * deltaTime;
942       rotaPiramides += rotaPiramidesOffset * deltaTime;
943
944       // 1. Acumulamos el tiempo continuo
945       tiempoTotal += deltaTime;
946       float velocidad = 0.01f; // Controla la duración de todo el ciclo
947
948       // 2. POSICIÓN: Onda amplificada y recortada (crea el movimiento y las pausas solas)
949       float onda = sin(tiempoTotal * velocidad);
950       separaPiramides = glm::clamp(onda * 2.5f, 0.0f, 2.0f);
951
952       // 3. INTERRUPTOR MATEMÁTICO: glm::step devuelve 0 o 1 sin usar 'if'
953       // cos() es positivo de ida y negativo de regreso.
954       // step(0, -cos) se convierte en 1.0 SOLO cuando la onda va de regreso.
955       float esRegreso = glm::step(0.0f, -cos(tiempoTotal * velocidad));
```

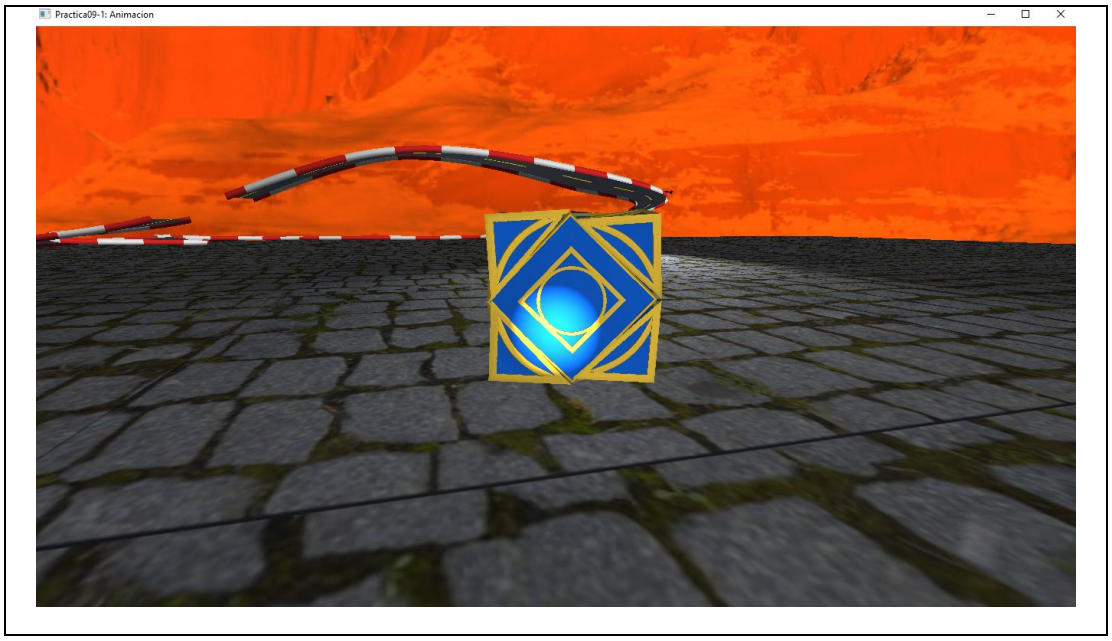
- Dibujo del holocron:

```
1468        // Holocron
1469        // Centro del holocron
1470        model = glm::mat4(1.0);
1471        model = glm::translate(model, glm::vec3(-20.0f, 5.0f, 0.0f));
1472        model = glm::rotate(model, rotaHolo, glm::vec3(0.0f, 1.0f, 0.0f));
1473        modelaux6 = model;
1474        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
1475        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1476        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1477        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1478        centroHolocron.RenderModel();
1479
1480        // Piramide 1
1481        model = modelaux6;
1482        model = glm::translate(model, glm::vec3(1.95f, 1.9f, 1.95f));
1483        model = glm::translate(model, glm::vec3(separaPiramides, separaPiramides, separaPiramides));
1484        model = glm::rotate(model, rotaPiramides, glm::vec3(1.0f, 1.0f, 1.0f));
1485        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
1486        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1487        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1488        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1489        piramide1.RenderModel();
1490
1491        // Piramide 2
1492        model = modelaux6;
1493        model = glm::translate(model, glm::vec3(1.95f, 1.95f, -1.95f));
1494        model = glm::translate(model, glm::vec3(separaPiramides, separaPiramides, -separaPiramides));
1495        model = glm::rotate(model, -rotaPiramides, glm::vec3(-1.0f, -1.0f, 1.0f));
1496        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
1497        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1498        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1499        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1500        piramide2.RenderModel();
1501
1502        // Piramide 3
1503        model = modelaux6;
1504        model = glm::translate(model, glm::vec3(-1.95f, 1.9f, 1.95f));
1505        model = glm::translate(model, glm::vec3(-separaPiramides, separaPiramides, separaPiramides));
1506        model = glm::rotate(model, -rotaPiramides, glm::vec3(1.0f, -1.0f, -1.0f));
1507        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
1508        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1509        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1510        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1511        piramide3.RenderModel();
1512
1513        // Piramide 4
1514        model = modelaux6;
1515        model = glm::translate(model, glm::vec3(-1.95f, 1.95f, -1.95f));
1516        model = glm::translate(model, glm::vec3(-separaPiramides, separaPiramides, -separaPiramides));
1517        model = glm::rotate(model, rotaPiramides, glm::vec3(-1.0f, 1.0f, -1.0f));
1518        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
1519        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1520        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1521        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1522        piramide4.RenderModel();
1523
1524        // Piramide 5
1525        model = modelaux6;
1526        model = glm::translate(model, glm::vec3(-1.95f, -1.9f, 1.95f));
1527        model = glm::translate(model, glm::vec3(-separaPiramides, -separaPiramides, separaPiramides));
1528        model = glm::rotate(model, -rotaPiramides, glm::vec3(-1.0f, -1.0f, 1.0f));
1529        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
1530        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1531        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1532        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1533        piramide5.RenderModel();
1534
1535        // Piramide 6
1536        model = modelaux6;
1537        model = glm::translate(model, glm::vec3(-1.95f, -1.9f, -1.95f));
1538        model = glm::translate(model, glm::vec3(-separaPiramides, -separaPiramides, -separaPiramides));
1539        model = glm::rotate(model, -rotaPiramides, glm::vec3(-1.0f, -1.0f, -1.0f));
1540        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
1541        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1542        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1543        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1544        piramide6.RenderModel();
```

```
1546        // Piramide 7
1547        model = modelaux6;
1548        model = glm::translate(model, glm::vec3(1.95f, -1.9f, 1.9f));
1549        model = glm::translate(model, glm::vec3(separaPiramides, -separaPiramides, separaPiramides));
1550        model = glm::rotate(model, rotaPiramides, glm::vec3(-1.0f, 1.0f, -1.0f));
1551        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
1552        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1553        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1554        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1555        piramide7.RenderModel();
1556
1557        // Piramide 8
1558        model = modelaux6;
1559        model = glm::translate(model, glm::vec3(1.95f, -1.9f, -1.95f));
1560        model = glm::translate(model, glm::vec3(separaPiramides, -separaPiramides, -separaPiramides));
1561        model = glm::rotate(model, -rotaPiramides, glm::vec3(1.0f, -1.0f, -1.0f));
1562        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
1563        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1564        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1565        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1566        piramide8.RenderModel();
```

Agrego captura de pantalla del holocron:

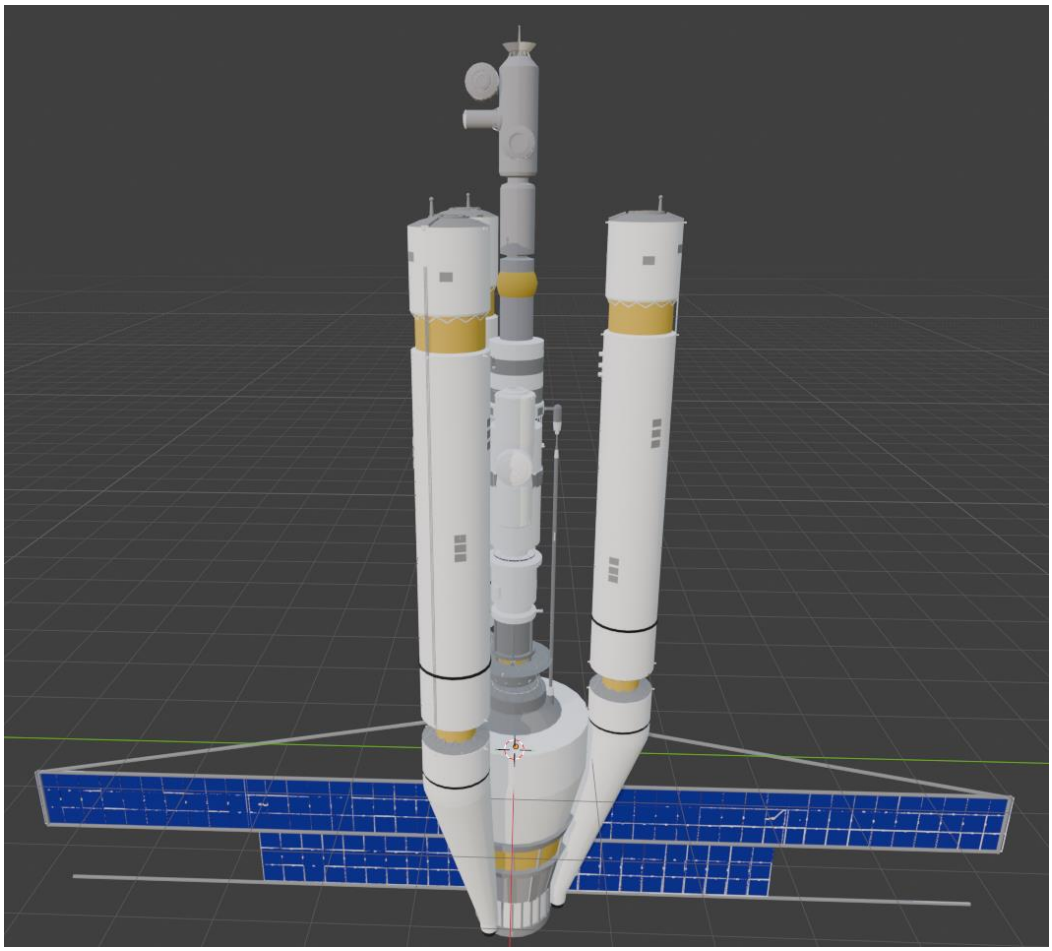




NOTA1 Sólo se agrega una captura de pantalla muy general, ya que todo se muestra en el video.

1.2 El cohete espacial despega y sigue una trayectoria a partir de ecuaciones y algoritmos.

Primero texturicé en Blender mi modelo del cohete, ya que cambié el modelo que puse en el previo porque de una vez quise texturizar el cohete que utilizaré en mi proyecto final. Después de texturizar, exporté mi cohete:



AL igual que con el holocron, le pedí ayuda a Google Gemini para que me ayudara a hacer el recorrido que tenía en mente. El prompt utilizado fue el siguiente:

"Quiero que mi cohete suba 20 unidades, para que después haga un recorrido gradual en forma de un cuarto de circunferencia hacia la derecha hasta que esté totalmente volteado verticalmente, posteriormente quiero que haga un recorrido circular infinito hacia la derecha."

Una vez obtenido el código, me puse a ajustar los valores de altura y velocidad para que se viera de mejor manera la animación.

Finalmente n *P09-2_320152896.cpp* modifiqué los bloques de código siguientes:

- **Carga de modelo:**

```
189   // Cohete
190
191   Model cohete;
```

```
576       // Cohete
577       cohete = Model();
578       cohete.LoadModel("Models/naveHailMary.obj");
```

- **Animación compleja:**

```
119   // Variables para el cohete
120   int estadoCohete = 0;
121   float movYCohete = 0.0f;
122   float anguloCurvaCohete = 0.0f;
123   float velInfinito = 0.01f;
124   float anguloInfinitoCohete = 0.0f;
```

```
972       // Animación del Cohete
973       float velCoheteLineal = 0.05f;
974       float velCoheteAngular = 0.005f;
975       float radioCurva = 20.0f;
976       float radioInfinito = 100.0f;
977
978       glm::vec3 posCohete(150.0f, 11.5f, -75.0f);
979       float rotCoheteZ = 0.0f;
980       float rotCoheteY = 0.0f;
981
982       if (estadoCohete == 0) {
983           // Fase 0: Ascenso de 20 unidades
984           movYCohete += velCoheteLineal * deltaTime;
985           if (movYCohete >= 75.0f) {
986               movYCohete = 75.0f;
987               estadoCohete = 1;
988           }
989           posCohete.y += movYCohete;
990       }
991       else if (estadoCohete == 1) {
992           // Fase 1: Cuarto de circunferencia a la derecha (plano XY)
993           posCohete.y += 75.0f;
994           anguloCurvaCohete += velCoheteAngular * deltaTime;
995
996           if (anguloCurvaCohete >= (90.0f * toRadians)) {
997               anguloCurvaCohete = 90.0f * toRadians;
998               estadoCohete = 2;
999           }
```

```
            posCohete.x += radioCurva * (1.0f - cos(anguloCurvaCohete));
            posCohete.y += radioCurva * sin(anguloCurvaCohete);
            rotCoheteZ = -glm::degrees(anguloCurvaCohete);
        }
        else if (estadoCohete == 2) {
            // Fase 2: Recorrido circular infinito (Órbita en plano XZ)
            posCohete.y += 75.0f + radioCurva;
            float xBase = 150.0f + radioCurva;

            anguloInfinitoCohete += velInfinito * deltaTime;

            // Ecuaciones paramétricas del círculo
            posCohete.x = xBase + radioInfinito * sin(anguloInfinitoCohete);
            posCohete.z = -75.0f - radioInfinito * (1.0f - cos(anguloInfinitoCohete));

            rotCoheteZ = -90.0f;

            // 2. Empareja directamente el Yaw (rotación en Y) con el ángulo orbital para que sea tangente
            rotCoheteY = glm::degrees(anguloInfinitoCohete);
        }
```

```
        // Cohete
        model = glm::mat4(1.0f);
        model = glm::translate(model, posCohete);
        model = glm::rotate(model, glm::radians(rotCoheteY), glm::vec3(0.0f, 1.0f, 0.0f));
        model = glm::rotate(model, glm::radians(rotCoheteZ), glm::vec3(0.0f, 0.0f, 1.0f));
        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
        model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        cohete.RenderModel();
```

2. **Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.**

   Para esta práctica no se presentaron problemas. Lo único que me gustaría aclarar nuevamente es el uso de Google Gemini.

3. **Conclusión:**
   a. **Los ejercicios del reporte:** Me gustó poder recrear de alguna manera el gif del holocron que vimos desde inicios del semestre, ya que me hace pensar en lo mucho que he aprendido.

   b. **Comentarios generales:** Considero que fueron ejercicios al nivel de lo que se enseñó en el laboratorio y sirvió para reforzar lo relacionado a la animación compleja.

   c. **Conclusión.** Esta práctica me sirvió mucho para saber cómo funciona de una mejor manera la animación en OpenGL. Crep que la mejor forma de animación y la más realista es cuando fusionamos partes de animación simple y de animación compleja, como lo puede ser el uso de if/else y funciones o algoritmos que simulen comportamientos físicos como vuelos o desplazamientos siguiendo ciertas curvas. Finalmente esto me ayuda mucho para saber como satisfacer los requerimientos de la animación del proyecto final.

Referencias bibliográficas.

https://sketchfab.com/3d-models/project-hail-mary-ship-6ab223b7be274f7ab473ca4aafa8a20d